UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Tonalá

Materia: Computo en la nube

Tema: Proyecto Final

Nombre: Alondra Guadalupe Armenta Gómez

Código: 217475274

Maestro: Víctor Hugo Vega Fregoso

CloudGallery

Guía de Despliegue:

1. Creacion de bucket S3

```
eee_k_5848769@runweb226376:~$ aws s3 mb s3://cloudgallery-bucket-12345
make_bucket: cloudgallery-bucket-12345
```

2. Crear EC2 desde CLI

   2.1 Creación de key

```
eee_k_5848769@runweb226376:~$ aws ec2 create-key-pair \
>   --key-name cloudgallery-key \
>   --query 'KeyMaterial' \
>   --output text > cloudgallery-key.pem
```

   2.2 Dar permisos al archivo

```
eee_k_5848769@runweb226376:~$ chmod 400 cloudgallery-key.pem
```

   Creación de instancia exitosa

```
eee_k_5848769@runweb226376:~$ aws ec2 run-instances \
>   --image-id ami-0c02fb55956c7d316 \
>   --count 1 \
>   --instance-type t2.micro \
>   --key-name cloudgallery-key \
>   --security-groups default
{
    "Groups": [],
    "Instances": [
        {
            "AmiLaunchIndex": 0,
            "ImageId": "ami-0c02fb55956c7d316",
            "InstanceId": "i-0e39cc2b377b3c545",
            "InstanceType": "t2.micro",
            "KeyName": "cloudgallery-key",
            "LaunchTime": "2026-05-07T19:28:45+00:00",
            "Monitoring": {
                "State": "disabled"
            },
            "Placement": {
                "AvailabilityZone": "us-east-1a",
                "GroupName": "",
                "Tenancy": "default"
            },
            "PrivateDnsName": "ip-172-31-43-92.ec2.internal",
            "PrivateIpAddress": "172.31.43.92",
            "ProductCodes": [],
            "PublicDnsName": "",
            "State": {
```

3. Crear Flask App

   3.1 Creación de la carpeta app

```
eee_k_5848769@runweb226376:~$ mkdir app
eee_k_5848769@runweb226376:~$ cd app
```

3.2 Crear un entorno virtual

```
eee_w_5848769@runweb226376:~/app$ python -m venv venv
eee_w_5848769@runweb226376:~/app$ venv\Scripts\activate
bash: venvScriptsactivate: command not found
eee_w_5848769@runweb226376:~/app$ source venv/bin/activate
(venv) eee_w_5848769@runweb226376:~/app$
```

3.2 Instalar Flask y Boto3

```
(venv) eee_w_5848769@runweb226376:~/app$ pip install flask boto3
Collecting flask
  Downloading https://files.pythonhosted.org/packages/9f/1a/8b6d48162861009d1e017a9740431c78d860809773b66cac220a11aa3310/Flask-2.2.5-
py3-none-any.whl (101kB)
    100% |[redacted]| 102kB 1.2MB/s
Collecting boto3
  Downloading https://files.pythonhosted.org/packages/df/23/9566acfb4665e261b936cbe89154c062d97a3bcfb1f8f2e54a51b2982f7b/boto3-1.33.1
3-py3-none-any.whl (139kB)
    100% |[redacted]| 143kB 1.3MB/s
Collecting werkzeug>=2.2.2 (from flask)
  Downloading https://files.pythonhosted.org/packages/f6/f8/9da63c1617ae2a1dec2fbf6412f3a0cfe9d4ce029eccbda6e1e4258ca45f/werkzeug-2.2
.3-py3-none-any.whl (233kB)
    100% |[redacted]| 235kB 1.4MB/s
Collecting itsdangerous>=2.0 (from flask)
  Downloading https://files.pythonhosted.org/packages/68/5f/447e04e828f47465eeab35b5d408b7ebaaaee207f48b7136c5a7267a30ae/itsdangerous
-2.1.2-py3-none-any.whl
Collecting importlib-metadata>=3.6.0; python_version < "3.10" (from flask)
  Downloading https://files.pythonhosted.org/packages/ff/94/64287b38c7de4c90683630338cf28f129decbba0a44f0c6db35a873c73c4/importlib_me
tadata-6.7.0-py3-none-any.whl
Collecting click>=8.0 (from flask)
  Downloading https://files.pythonhosted.org/packages/7e/d4/7ebdbd03970677812aac39c869717059dbb71a4cfc033ca6e5221787892c/click-8.1.8-
py3-none-any.whl (98kB)
    100% |[redacted]| 102kB 1.3MB/s
```

3.3 Crear archivo .txt

```
(venv) eee_w_5848769@runweb226376:~/app$ touch requirements.txt
(venv) eee_w_5848769@runweb226376:~/app$ echo "flask" > requirements.txt
(venv) eee_w_5848769@runweb226376:~/app$ echo "boto3" >> requirements.txt
```

4. Crear archivo .py

```
GNU nano 2.5.3                    File: app.py                    Modified

from flask import Flask, render_template, request
import boto3

app = Flask(__name__)

S3_BUCKET = "cloudgallery-bucket-12345"

s3 = boto3.client("s3")

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/upload", methods=["POST"])
def upload():

    file = request.files["file"]

    try:

        s3.upload_fileobj(
            file,
            S3_BUCKET,
            file.filename
```

5. Crear archivo html

```
(venv) eee_w_5848769@runweb226376:~/app$ touch templates/index.html
(venv) eee_w_5848769@runweb226376:~/app$ nano index.html
```

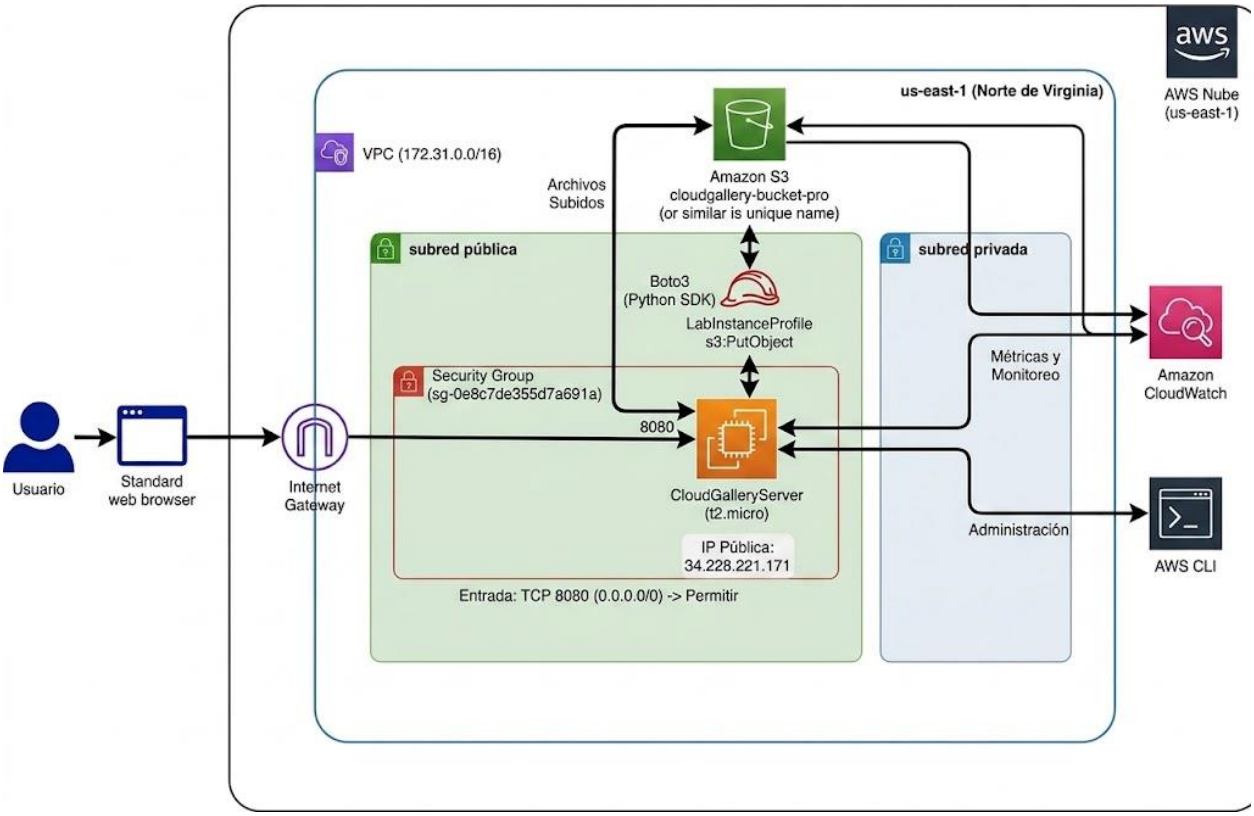
```
  GNU nano 2.5.3                    File: index.html

<!DOCTYPE html>
<html>

<head>

<title>CloudGallery</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body class="container mt-5">

<h1>CloudGallery</h1>

<form
    action="/upload"
    method="POST"
    enctype="multipart/form-data"
>

<input
    type="file"
    name="file"
```

6. Modificar el rol IAM

7. Ejecutar

```
[ec2-user@ip-172-31-43-92 app]$ python3 app.py
/home/ec2-user/.local/lib/python3.7/site-packages/boto3/compat.py:82: PythonDeprecationWarning: Boto3 will no longer support Python 3.7 starting Dec
ember 13, 2023. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.8 or later. More information can b
e found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://172.31.43.92:8080
Press CTRL+C to quit
187.204.121.37 - - [11/May/2026 04:04:49] "GET / HTTP/1.1" 200 -
187.204.121.37 - - [11/May/2026 04:04:49] "GET /favicon.ico HTTP/1.1" 404 -
187.204.121.37 - - [11/May/2026 04:05:37] "POST /upload HTTP/1.1" 200 -
187.204.121.37 - - [11/May/2026 04:05:55] "POST /upload HTTP/1.1" 200 -
187.204.121.37 - - [11/May/2026 04:06:02] "POST /upload HTTP/1.1" 200 -
187.204.121.37 - - [11/May/2026 04:06:45] "POST /upload HTTP/1.1" 200 -
```

Diagrama de Arquitectura:



Justificación de Decisiones:

EC2: Utilice EC2 por el control que brinda sobre el entorno de ejecución y el sistema operativo, ademas es un servicio flexible para instalar dependencias de Python, y EC2 permite mantener una conexión persistente y facilita la depuración en tiempo real durante la fase de desarrollo.

S3: Utilice este servicio para la creación de un bucket de S3 como almacenamiento primario de archivos, este cuenta con una escalabilidad automática, es mas eficiente que guardar archivos en el disco duro de EC2, garantiza que las imágenes no se pierdan si el servidor falla.

IAM Roles: Utilice este servicio para dar accesos, en lugar de Access Keys estáticas, esta es una mejor practica de seguridad en AWS, ya que asigna un rol directamente a la EC2 a el SDK, esto elimina el riesgo de filtrar llaves maestras en el código fuente

AWS CLI: Este servicio lo utilice para la implementación de comandos ya que permite realizar tareas de creación, mantenimiento, elimiacion, entre otras, además facilita la automatización de procesos mediante scripts de bash.

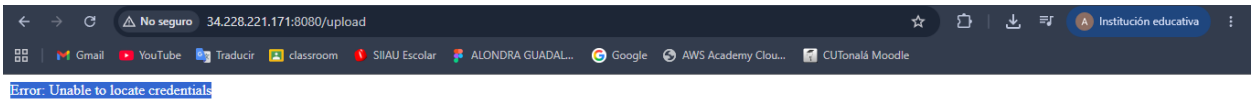
Amazon CloudWatch: es un servicio que te ayuda a el monitoreo de recursos, es de gran importancia para observar el comportamiento de la instancia de EC2 y el crecimiento del bucket S3

Análisis de Costos (Estimado):

Servicio	Uso	Unidades	Costo unitario	Total Mensual
Amazon EC2	Instancia t2.micro (Linux)	730 horas	$0.0116 / hora	$8.47
Amazon S3	Almacenamiento estándar	10 GB	$0.023 / GB	$0.23
Amazon S3	Peticiones de subida (PUT)	1,000 req.	$0.005 / 1,000	$0.01
CloudWatch	Métricas y Dashboards	3 métricas	Capa gratuita	$0.00
Data Transfer	Transferencia hacia Internet	5 GB	$0.09 / GB	$0.45
Total Mensual				**$9.16 USD**

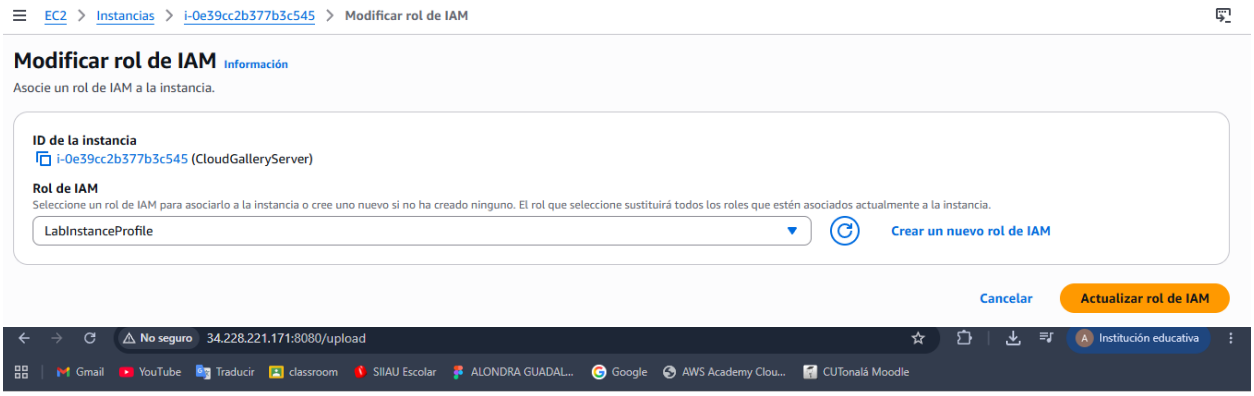
Pruebas de Resiliencia

El usuario intenta subir algún archivo y le sale un error "*Error: Unable to locate credentials*"



Error: Unable to locate credentials

Esto podría ser un problema de roles del servicio IAM
Para resolverlo solo debemos de monitorear los roles de ese servicio y confirmar que tenga permisos de "LabInstanceProfile"



Archivo subido con éxito

Una aplicación web sencilla donde:

- subes imágenes,
- se guardan en S3,
- el backend corre en EC2,
- usas Python + Boto3,
- usas AWS CLI,
- puedes usar MinIO localmente para pruebas,
- y toda la infraestructura se crea con scripts.

Servicio	Uso
EC2	Backend Flask
S3	Almacenamiento imágenes
IAM	Permisos
CloudWatch	Logs básicos
AWS CLI	Automatización

```
cloudgallery/
|
 ├── app/
|   ├── app.py
|   ├── requirements.txt
|   ├── templates/
|   |   └── index.html
|   └── static/
|
 ├── scripts/
|   └── deploy.sh
|
 ├── docs/
|
└── README.md
```

```
from flask import Flask, render_template, request
import boto3

app = Flask(__name__)
```

```
# 1. Configuración de AWS
S3_BUCKET = "cloudgallery-bucket-12345"
s3 = boto3.client("s3")

# 2. Definición de Rutas (Deben ir ANTES del run)
@app.route("/")
def home():
    return render_template("index.html")

@app.route("/upload", methods=["POST"])
def upload():
    file = request.files["file"]
    try:
        s3.upload_fileobj(file, S3_BUCKET, file.filename)
        return "Archivo subido con éxito"
    except Exception as e:
        return f"Error: {str(e)}"

# 3. El arranque del servidor (SIEMPRE al final)
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=5005)
```